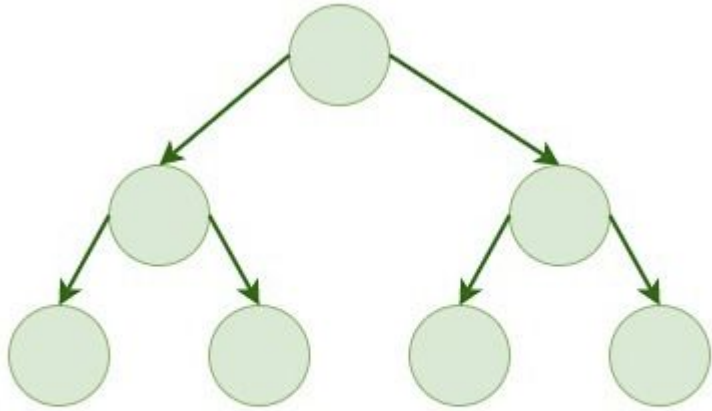
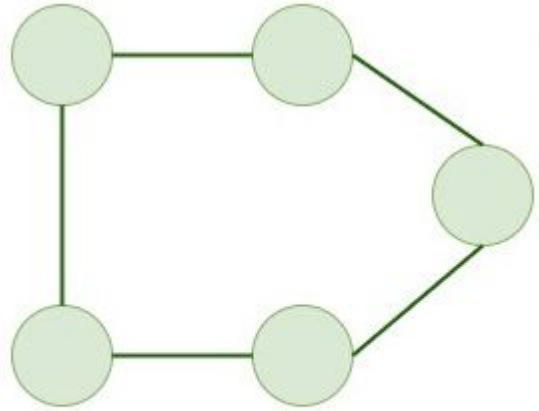


Graph Data Structure & Traversal Techniques

Graph



Tree



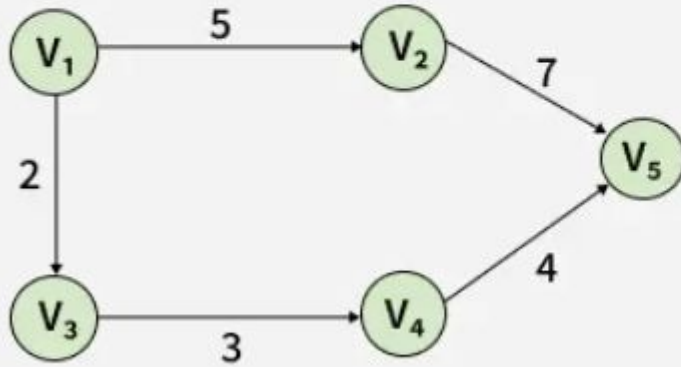
Graph

Graph

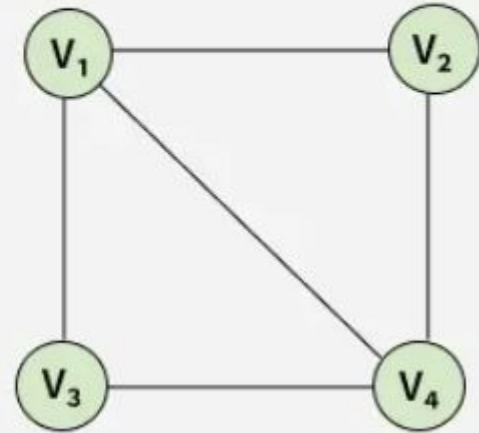
- Graph is a **non-linear data structure** like tree data structure.
- A Graph is composed of a set of vertices(V) and a set of edges(E). The vertices are connected with each other through edges.
- The limitation of tree is, it can only represent hierarchical data. For situations where nodes or vertices are randomly connected with each other, we use Graph.

Various Types of Graph

Weighted Graph

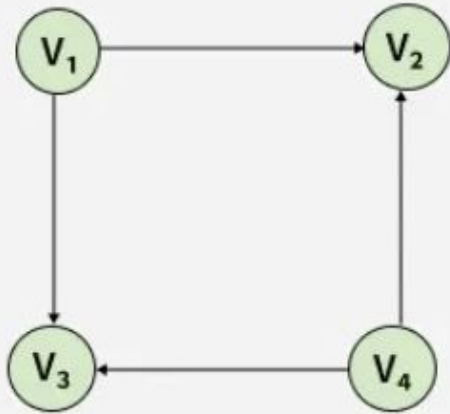


Unweighted Graph

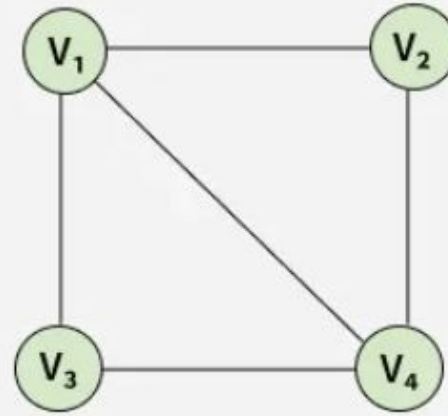


Various Types of Graph

Directed Graph



Undirected Graph

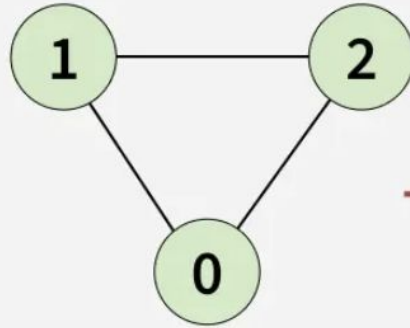


Representation of Graph

- 2 most common ways to represent a graph :
 - Adjacency Matrix
 - Adjacency List

Representation of Graph (Adjacency Matrix)

- create a 2D matrix $\text{adjMat}[n][n]$ having dimension $n \times n$.
- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as **1**.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as **0**.



Undirected Graph

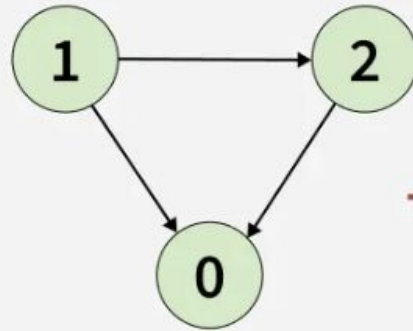


	0	1	2
0	0	1	1
1	1	0	1
2	1	1	0

Adjacency Matrix

Representation of Graph (Adjacency Matrix)

- create a 2D matrix $\text{adjMat}[n][n]$ having dimension $n \times n$.
- If there is an edge from vertex i to j , mark $\text{adjMat}[i][j]$ as **1**.
- If there is no edge from vertex i to j , mark $\text{adjMat}[i][j]$ as **0**.



Directed Graph

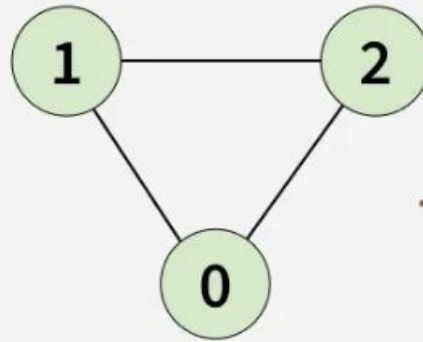


	0	1	2
0	0	0	0
1	1	0	1
2	1	0	0

Adjacency Matrix

Representation of Graph (Adjacency List)

- create an array of list of size n as `adjList[n]`.
- `adjList[0]` will have all the nodes which are connected (neighbour) to vertex 0.
- `adjList[1]` will have all the nodes which are connected (neighbour) to vertex 1 and so on



Undirected Graph

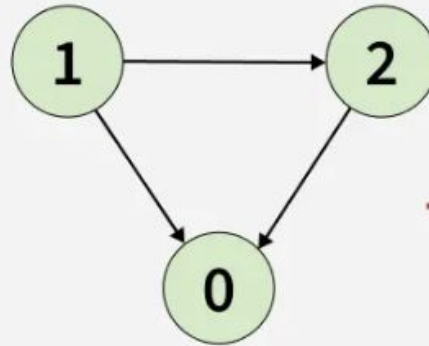


Vertex	Neighbour	
0 →	1	2
1 →	0	2
2 →	0	1

Adjacency List

Representation of Graph (Adjacency List)

- create an array of list of size n as `adjList[n]`.
- `adjList[0]` will have all the nodes which are connected (neighbour) to vertex 0.
- `adjList[1]` will have all the nodes which are connected (neighbour) to vertex 1 and so on



Directed Graph

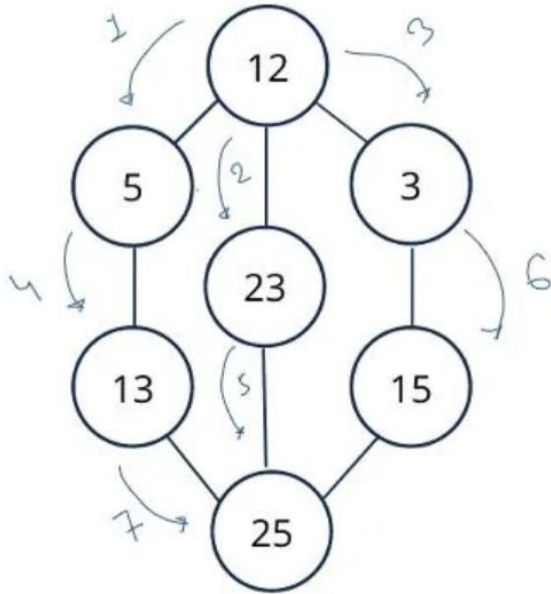


Vertex	Neighbour	
0	→	
1	→	0 2
2	→	0

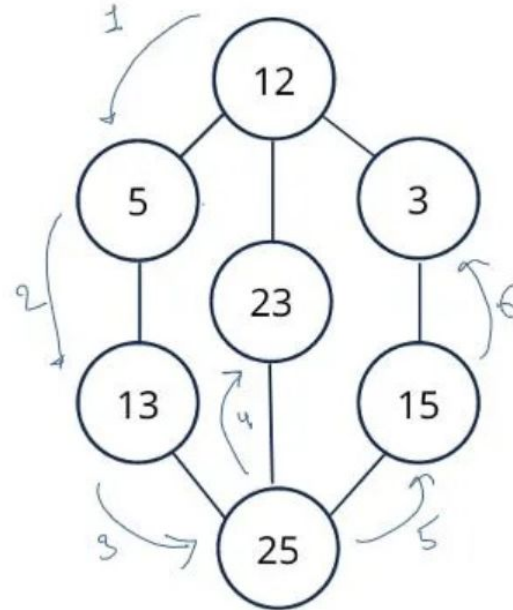
Adjacency List

Traversal Techniques (BFS & DFS)

BFS - BFS grows the tree level by level



DFS - DFS grows it sub-tree by sub-tree.



BFS (Breadth First Search)

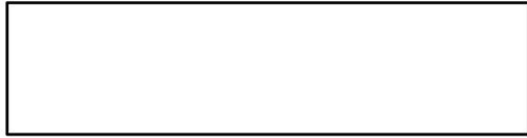
- Starting from any chosen starting point in a graph, BFS explores all the immediate neighbors (i.e., nodes at distance 1), then all nodes at distance 2, and so on.
- The key data structure used in BFS is a **queue (FIFO)**, which ensures nodes are explored in the order they are discovered.

Breadth First Search (BFS)

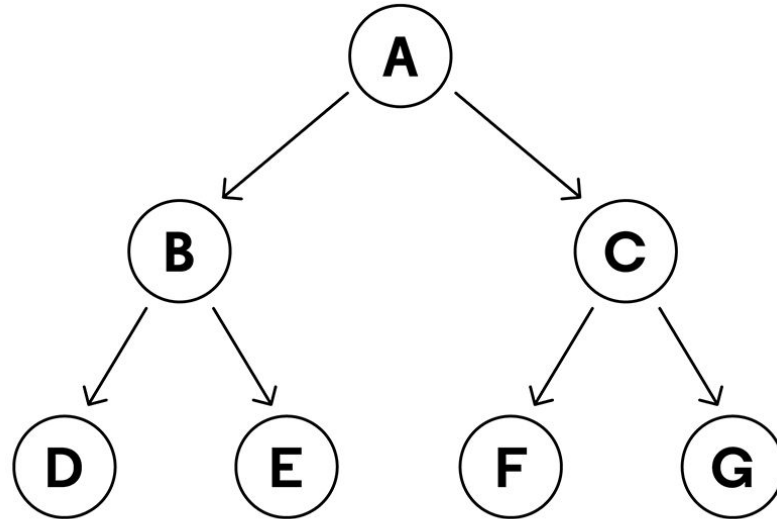
- Algorithm:
- Initialize a queue and a visited set (or list) to keep track of visited nodes
- Start by enqueueing the root node and marking it as visited
- While the queue is not empty:
 - Dequeue a node n from the front
 - Process node n (e.g., print it, check if it's the goal)
 - Enqueue all unvisited neighbors (or children in a tree) of node n and mark them as visited
- Repeat until all nodes are explored or the goal is found

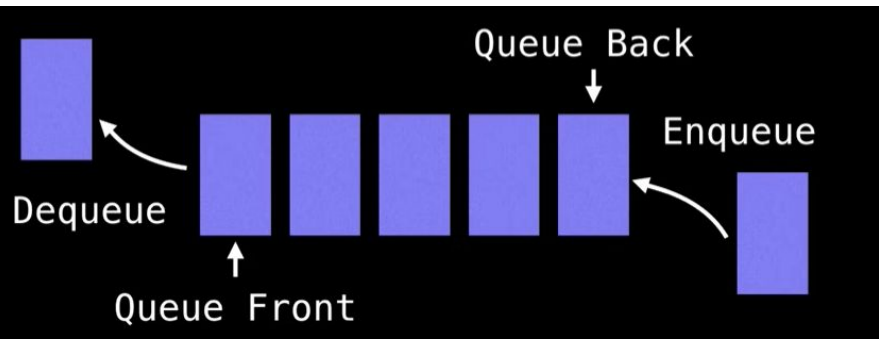
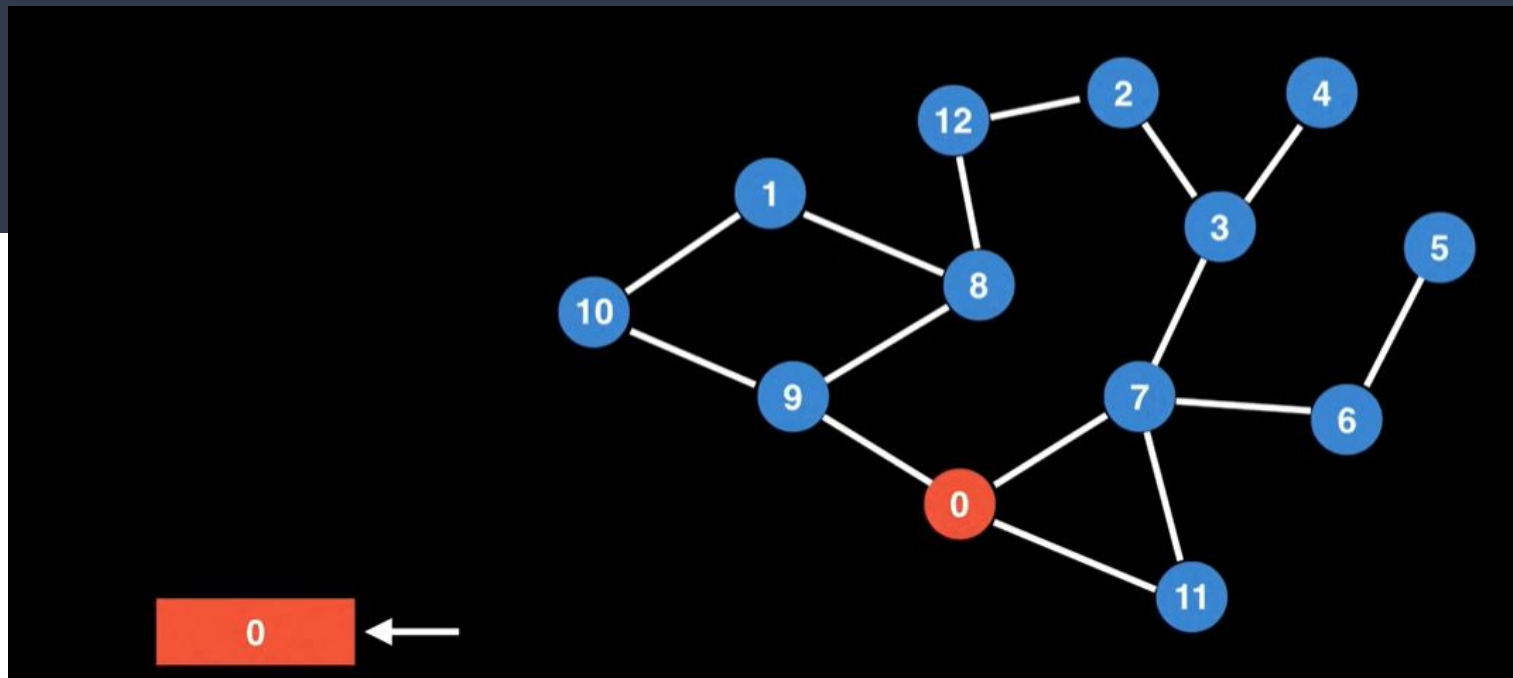
Breadth First Search (BFS)

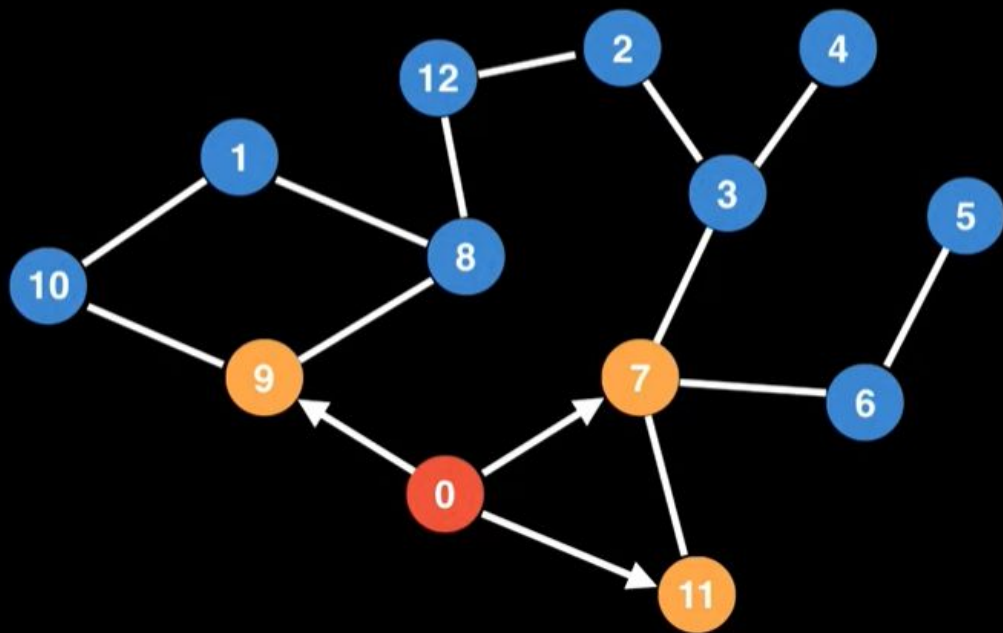
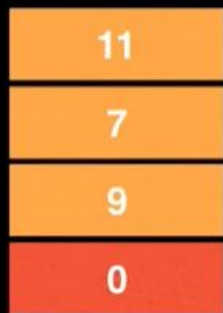
Tree with an Empty Queue

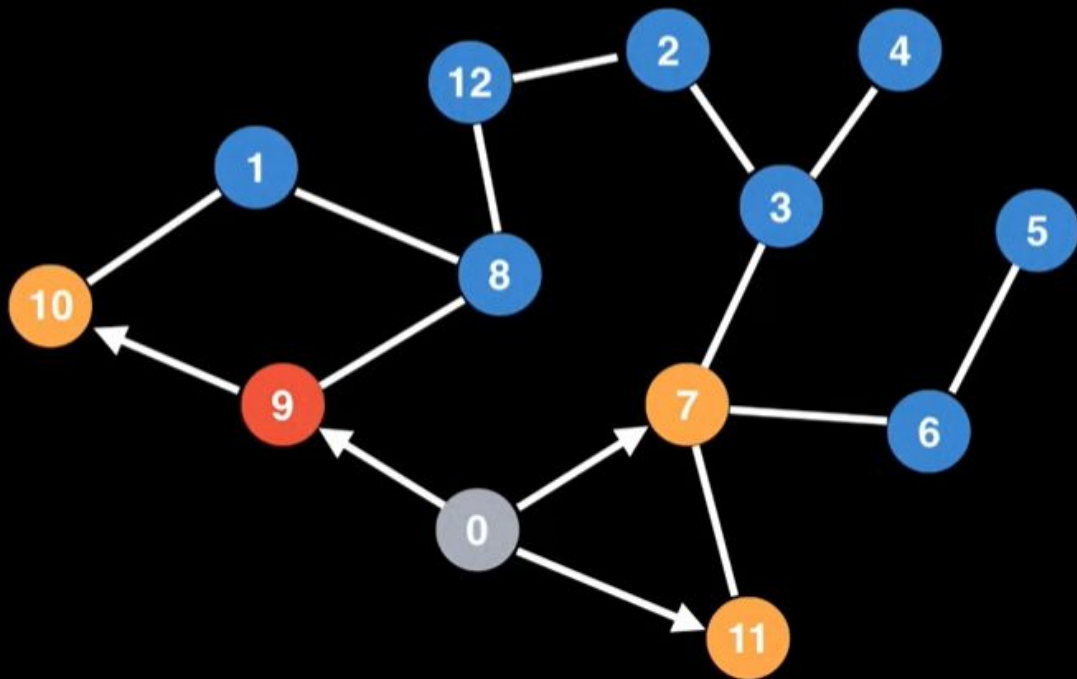
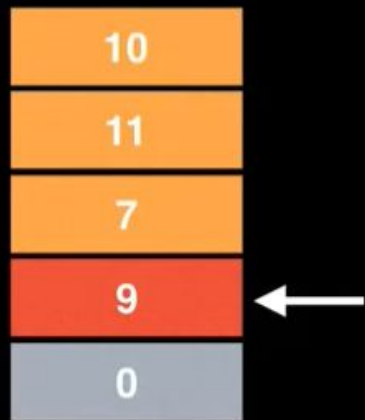


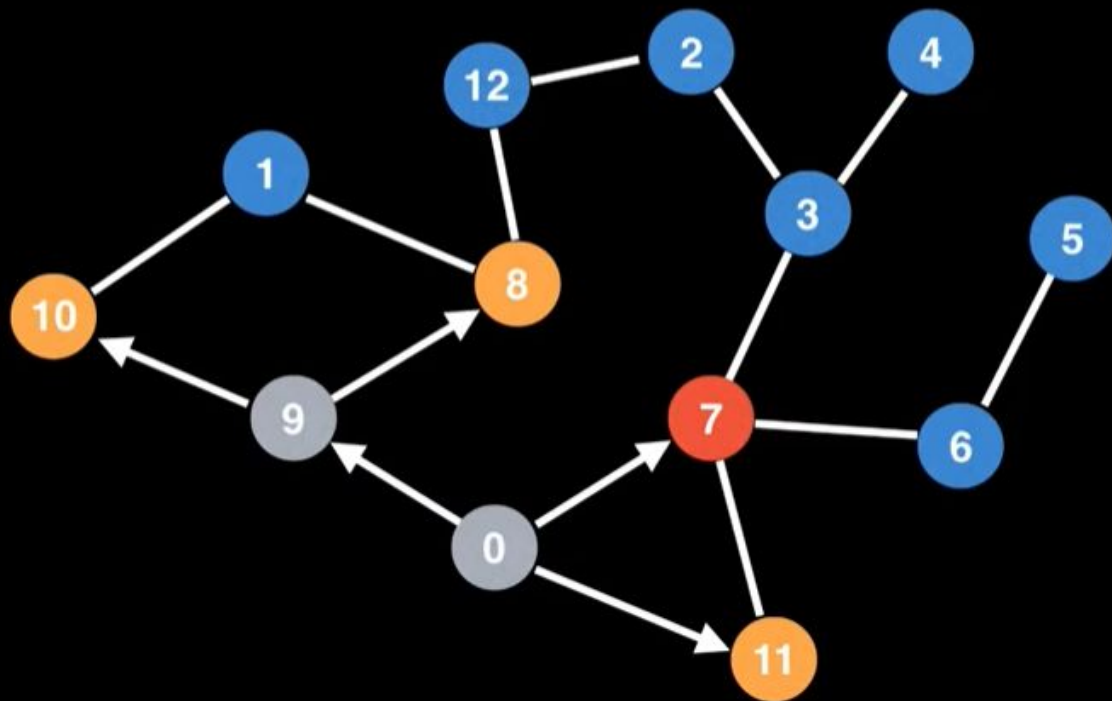
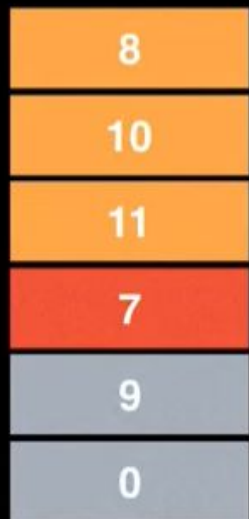
Frontier Queue
FIFO (First in First Out)

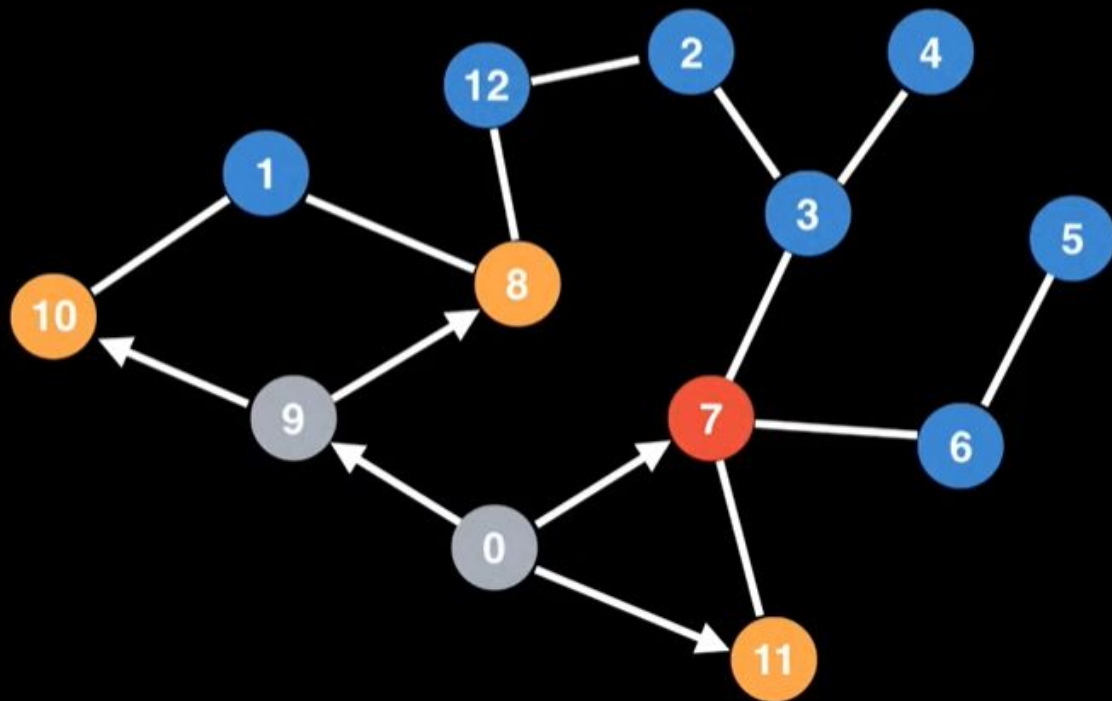
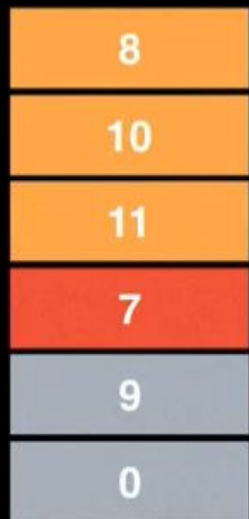




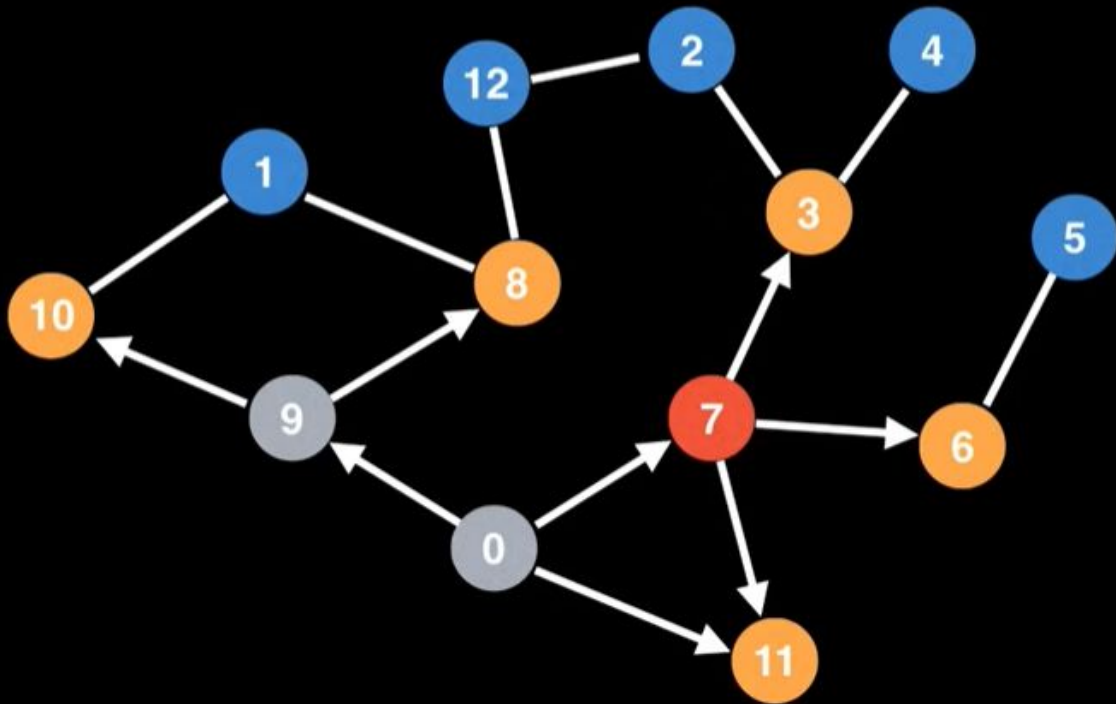


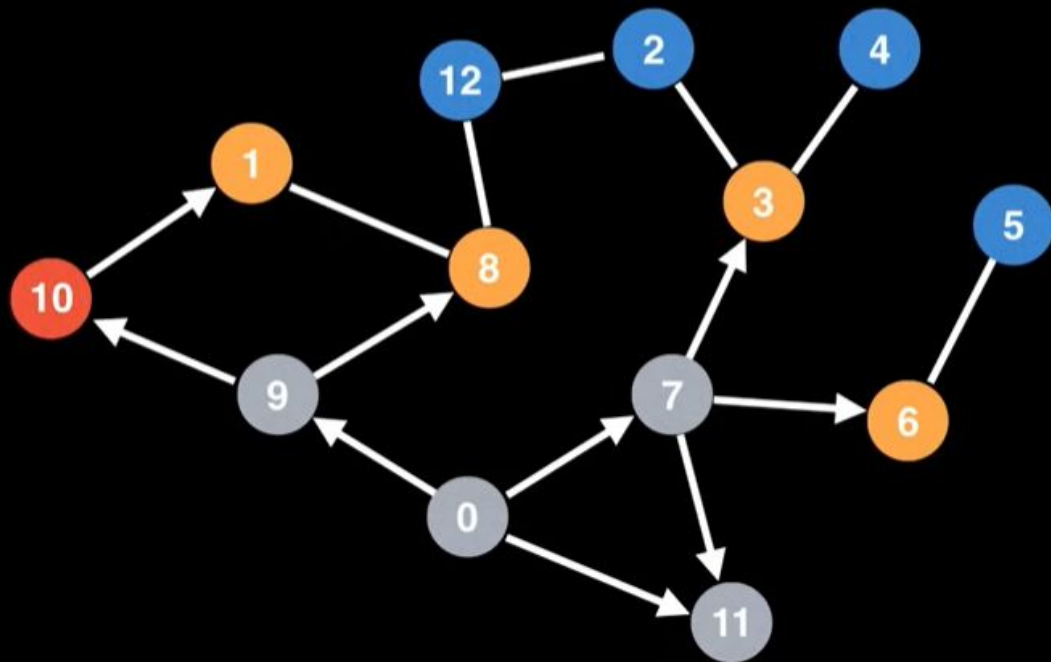


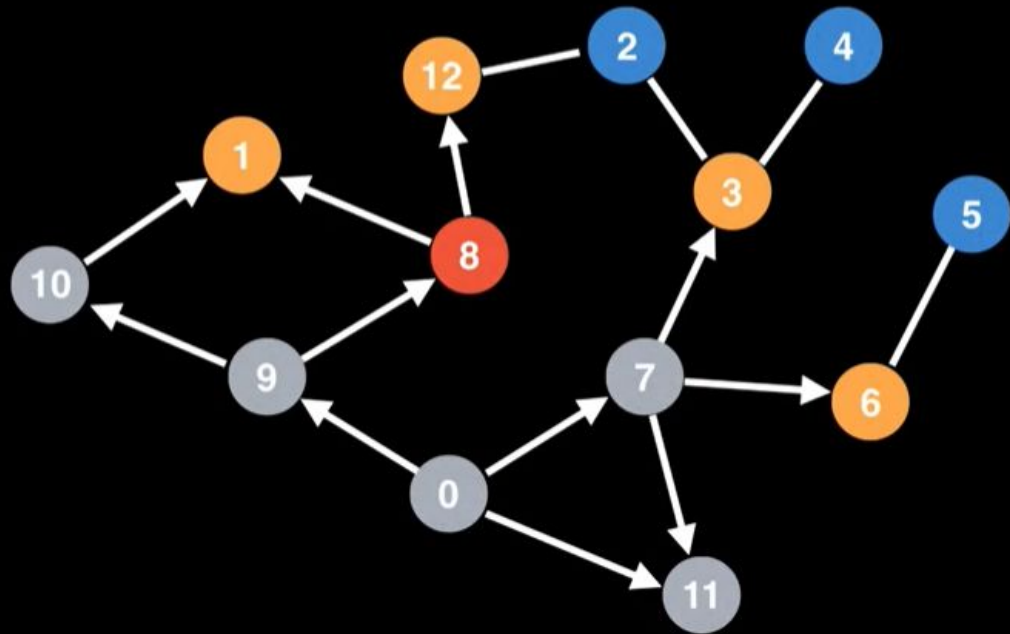




3
6
8
10
11
7
9
0







```
visited[start] = true;
q.push(start);

while (!q.empty()) {
    int node = q.front();
    q.pop();

    cout << node << " ";

    for (int i = 0; i < adj[node].size(); i++) {
        int next = adj[node][i];

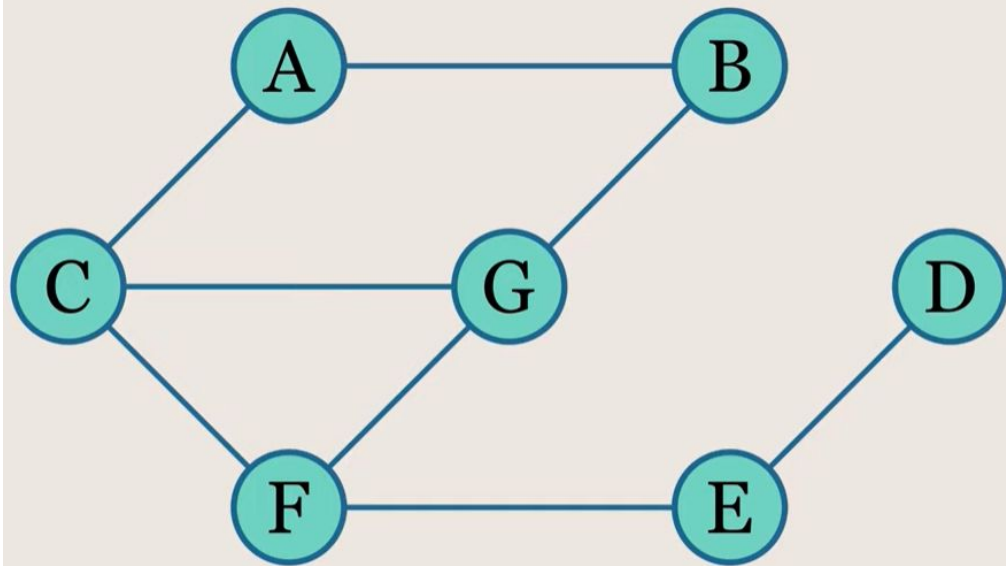
        if (!visited[next]) {
            visited[next] = true;
            q.push(next);
        }
    }
}
```

Depth-First Search (DFS)

- Depth-First Search (DFS) is a path-oriented traversal technique. It starts from any chosen node and explores as far down a branch as possible before backtracking.
- Unlike BFS, which explores nodes level by level, DFS dives deep into the tree or graph before exploring sibling nodes.
- DFS can be implemented in two primary ways:
 - Recursively (Implicit stack via function calls)
 - Iteratively (Explicit stack)
 - DFS uses a stack (LIFO) to determine the next node to visit, whether explicitly or through recursion.

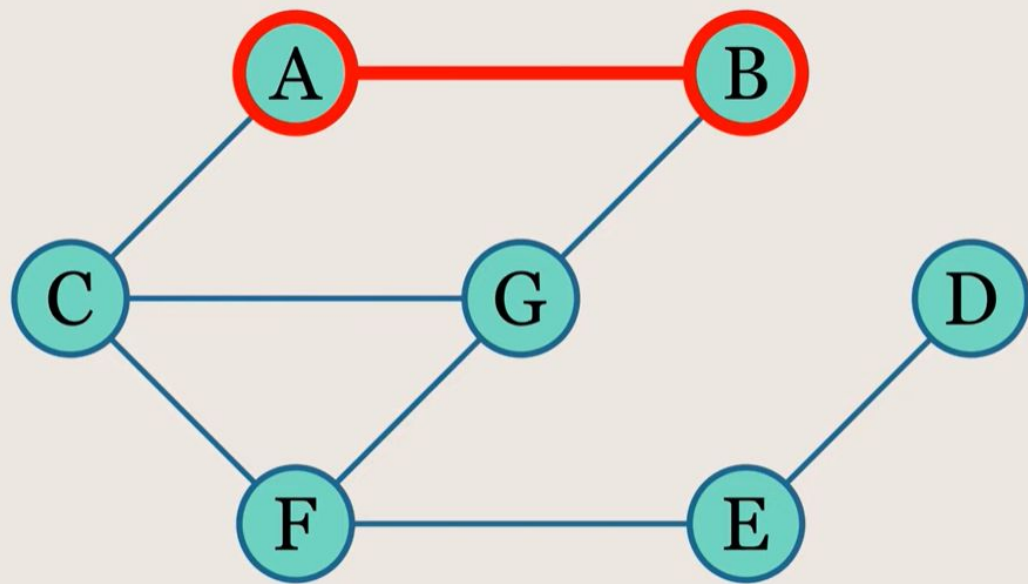
Depth-first search (DFS)

- Algorithm:
- Initialize a stack (or use recursion) and a visited set/list
- Push the root node onto the stack
- While the stack is not empty:
 - Pop the node from the top
 - If not already visited:
 - Process it (e.g., print or check if it's the goal)
 - Mark it as visited
 - Push its children onto the stack
- Repeat until the stack is empty or the goal is found



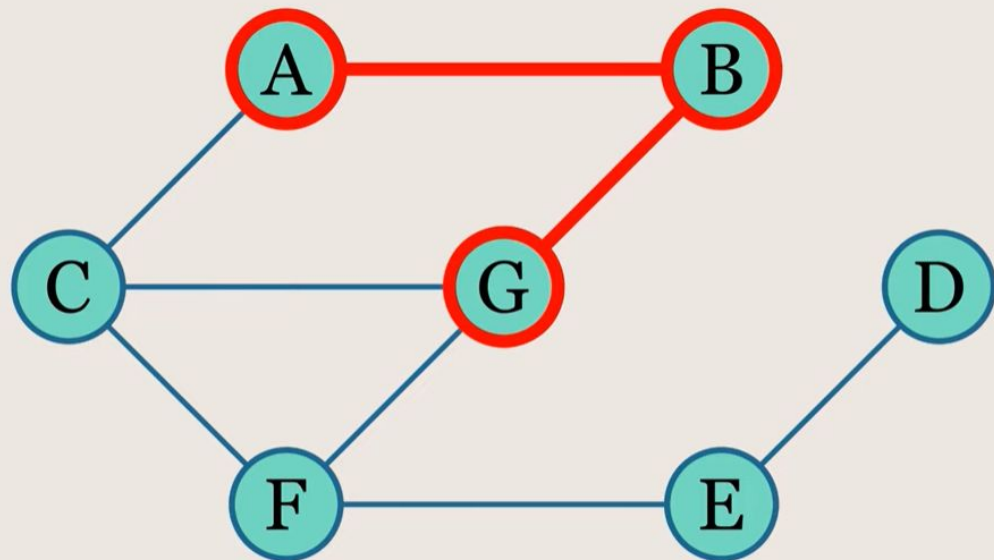
$\{\}$



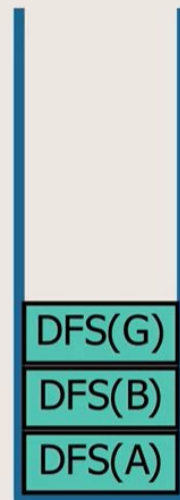


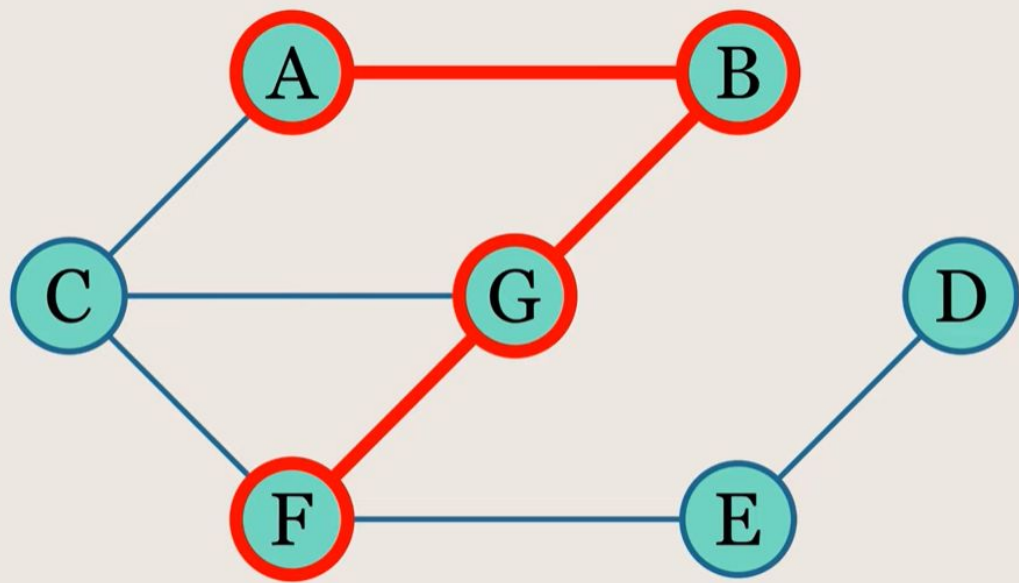
A B





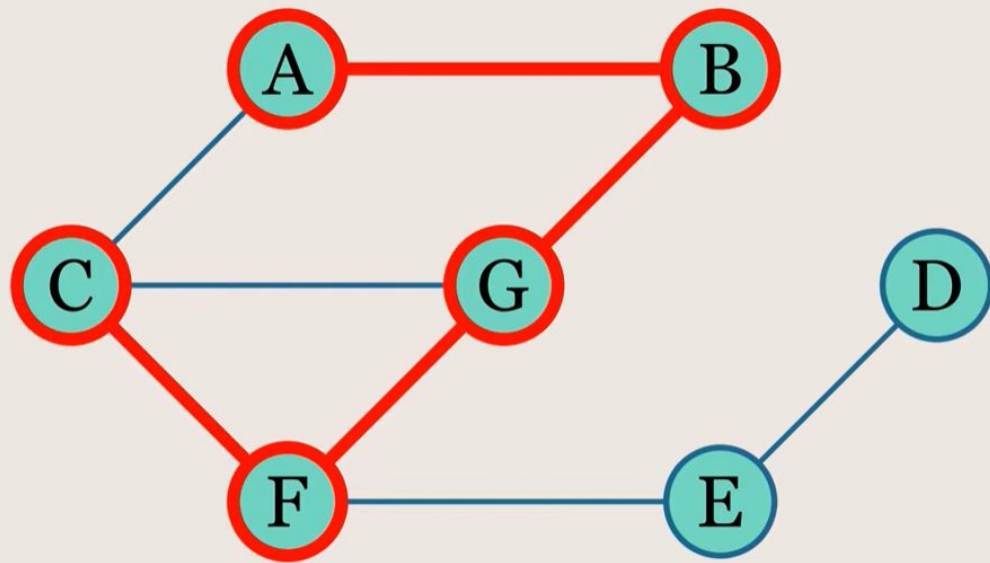
A B G



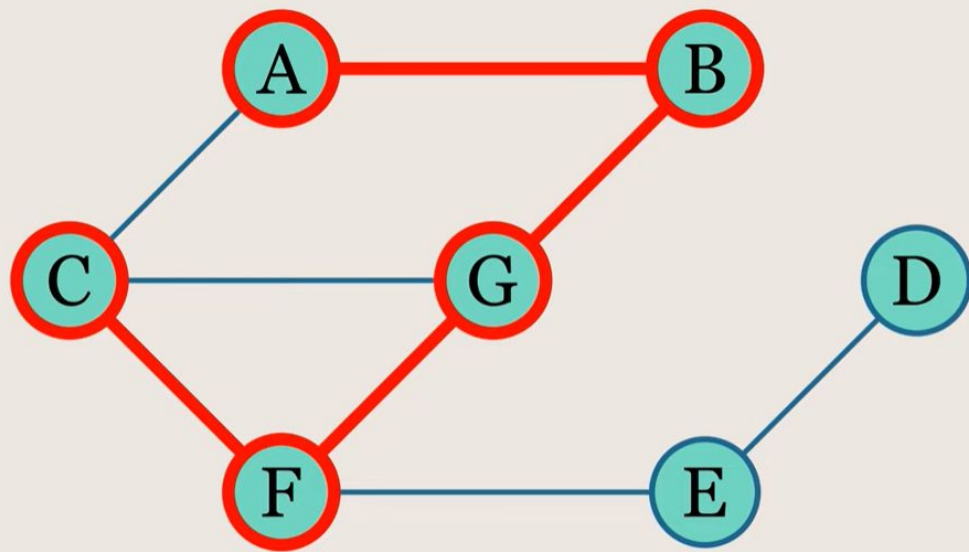


A B G F





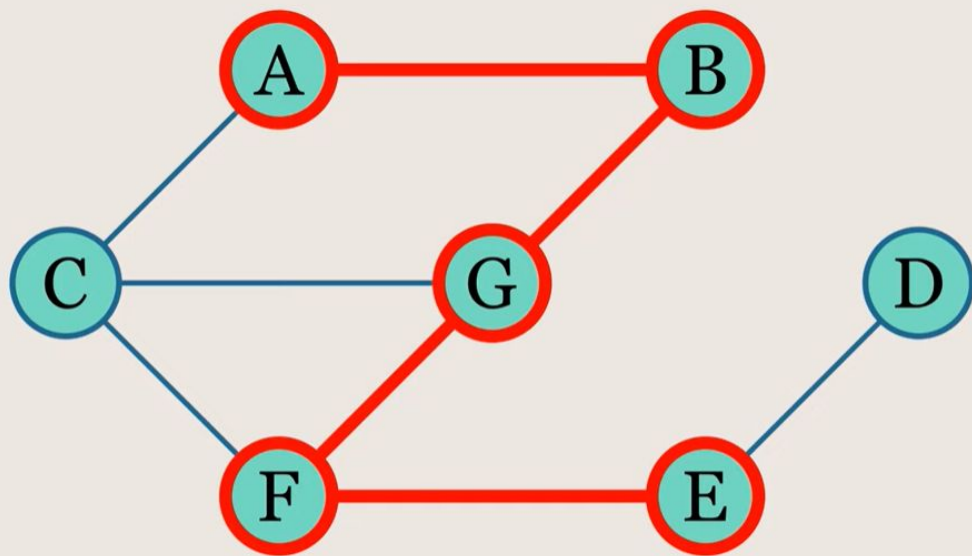
A B G F C



A B G F C

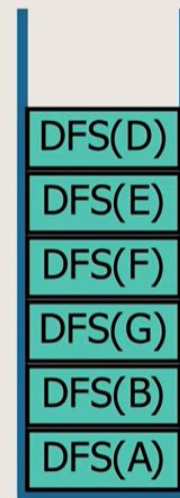
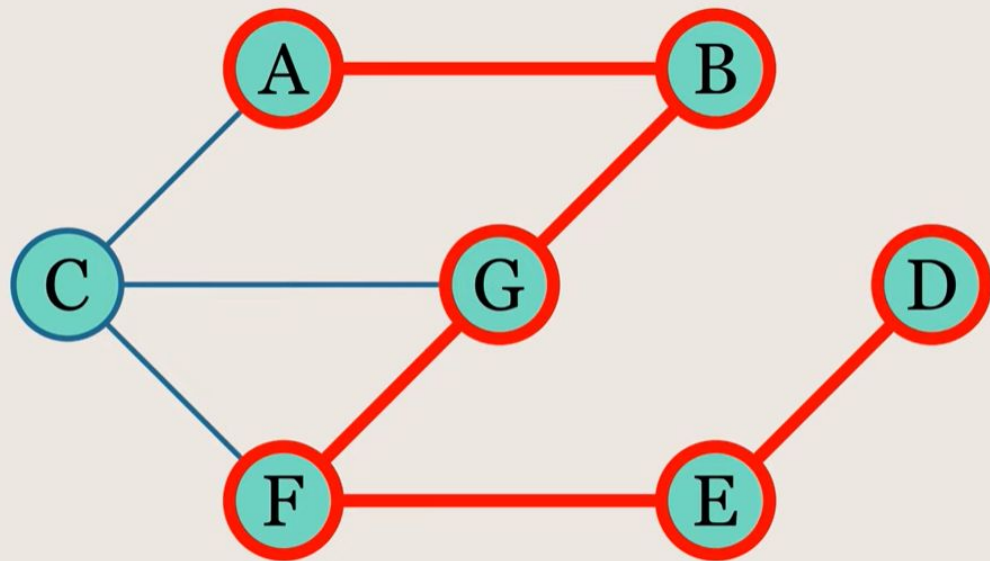
DFS(C)



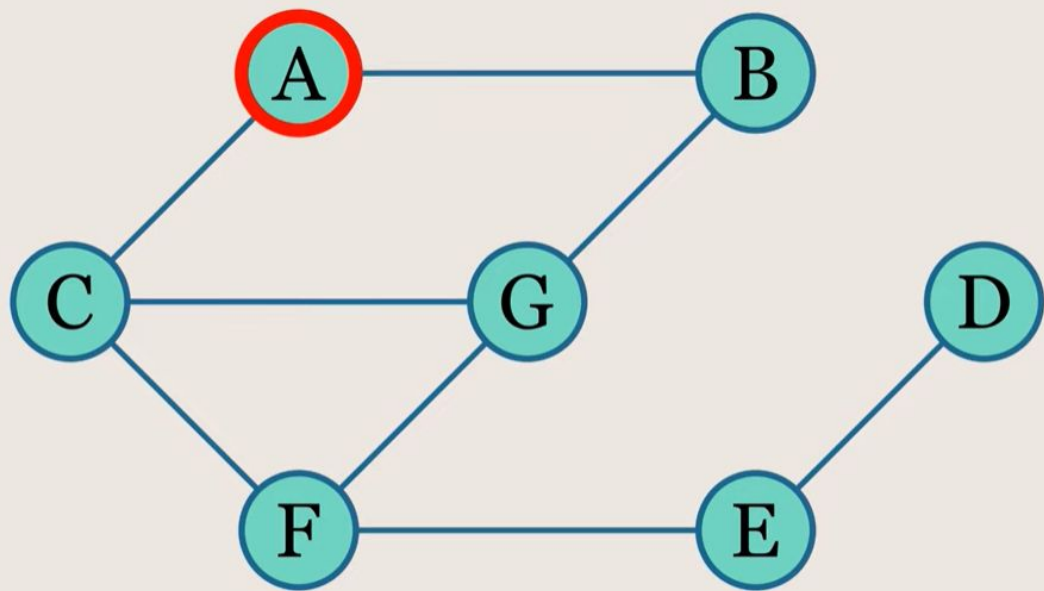


A B G F C E





A B G F C E D



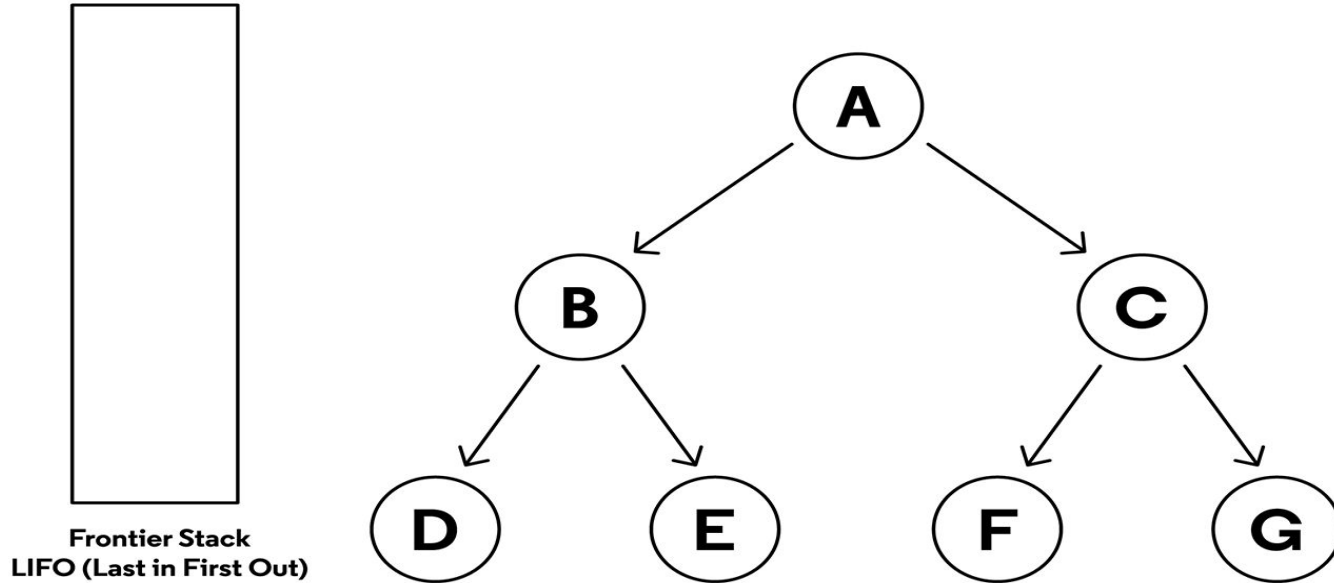
A B G F C E D

DFS(A)



Depth-first search (DFS)

Tree with an Empty Stack



References

- <https://www.geeksforgeeks.org/dsa/difference-between-graph-and-tree/>
- <https://www.geeksforgeeks.org/dsa/iterative-depth-first-traversal/>
- <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
-